MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS

DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATIC

REPORT ON LABORATORY WORK NO. 3

Operating Systems

The Command Line: Shell, Pipes and Text Filters

Done by: st. gr. FAF-243, Andrei Elciscev

Checked by:
Patricia Reitman, university assistant
DISA, FCIM, UTM

Chișinău, 2026

Contents

1 Laboratory Work 3: The Command Line **3**

1.1 Purpose of the Work . . . 3

1.2 Environment . . . 3

2 Part 1. Move Fast, Find Help **4**

2.1 Tab Completion . . . 4

2.2 Command History . . . 4

2.3 Built-in and External Commands . . . 4

2.4 Finding Help Without a Browser . . . 5

2.5 Observe Questions and Answers . . . 6

3 Part 2. What the Shell Does to Your Line **7**

3.1 Preparing the Test Directory . . . 7

3.2 Predictions and Shell Expansion . . . 7

3.3 Prediction Table . . . 8

3.4 The Filename Containing a Space . . . 9

3.5 Single Quotes, Double Quotes and Backslash . . . 10

3.6 Observe Questions and Answers . . . 10

4 Part 3. Redirection, Chaining and Jobs **11**

4.1 Standard Output and Standard Error . . . 11

4.2 Order of Redirections . . . 12

4.3 Input Redirection and Chaining . . . 13

4.4 The Operators ;, &&, and || . . . 13

4.5 Background Jobs . . . 14

4.6 Observe Questions and Answers . . . 15

5 Part 4. Answer Questions with One-Liners **16**

5.1 Dataset . . . 16

5.2 Q1. How Many Requests Failed? . . . 16

5.3 Q2. How Many Different Pages Were Requested, and Which? . . . 16

5.4 Q3. How Many Requests Did Each Page Get? . . . 17

5.5 Q4. Which User or Users Have the Most Failed Requests? . . . 17

5.6 Q5. How Many Requests Did user3 Make, and How Many Failed? . . . 18

5.7 Q6. Print the Last 3 Failed Requests, Showing Only Time and User . . . 19

5.8 Q7. Which Login Shells Appear in /etc/passwd, and How Many Accounts Use Each? 19
5.9 Q8. Why Do ls /etc | wc -l and ls -l /etc | wc -l Differ by Exactly One? . . . 20
5.10 Complete oneliners.sh Execution 21
5.11 Bonus: Failed Requests at Minutes Ending in Zero 22

6 Part 5. Your First Script **23**
6.1 Creating report.sh 23
6.2 Running the Script 24
6.3 Testing Error Handling 25
6.4 Observe Questions and Answers 25

7 Conclusions **26**

8 Appendix A. oneliners.sh **27**

9 Appendix B. report.sh **27**

1. Laboratory Work 3: The Command Line

1.1. Purpose of the Work

The purpose of this laboratory work was to develop practical skills for working efficiently with the Linux command line and Bash shell. The laboratory focused on shell command processing, command completion, command history, built-in and external commands, environment variables, manual pages, globbing, quoting, escaping, redirection, command chaining, background jobs, pipes, and text-processing utilities.

Another important objective was to learn how several small command-line tools can be combined into pipelines to answer practical questions about structured data. An artificial web access log was used as a common dataset, allowing commands such as `grep`, `cut`, `sort`, `uniq`, `wc`, `head`, and `tail` to be combined into one-line solutions.

The laboratory also introduced Bash scripting. The final tasks required the creation of an `oneliners.sh` file containing one-line solutions and a `report.sh` program that processes an access log and handles invalid input.

1.2. Environment

The laboratory was performed in a Linux environment using the Bash shell. The working directory was:

```
~/os-lab3
```

The artificial access log contained 500 lines. Each line had the following structure:

```
date time user result page
```

For example:

```
2026-10-05 10:01 user1 OK /page1
```

The dataset was generated using the following command:

```
mkdir -p ~/os-lab3 && cd ~/os-lab3
for i in $(seq 1 500); do
    echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -
    eq 0 ] && echo FAIL || echo OK) /page$((i % 4))"
done > access.log
head -3 access.log
wc -l access.log
```

The final command should report:

```
500 access.log
```

2. Part 1. Move Fast, Find Help

2.1. Tab Completion

The command:

```
ls /usr/sh
```

was entered followed by pressing the Tab key twice. Bash displayed or completed the available matching paths.

The command:

```
cd /et
```

was then followed by a single Tab press. Bash completed the path to the available directory, resulting in:

```
/etc
```

Tab completion is useful because it reduces typing and helps discover valid commands, files, and directories.

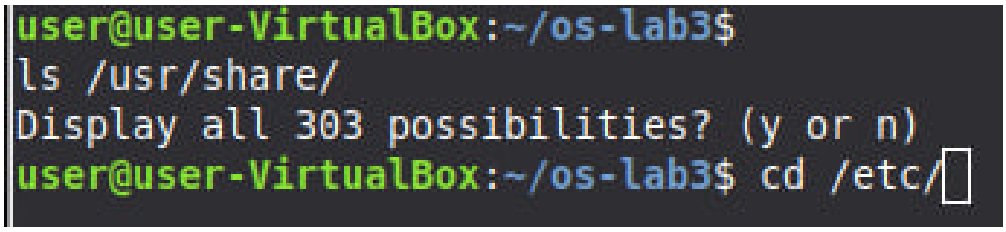


Figure 2.1: Tab completion in the Linux terminal

2.2. Command History

Three different commands were executed. Afterwards, `Ctrl+R` was pressed and several characters from the first command were entered. Bash searched the command history for the most recent command matching the entered text. Pressing Enter executed the displayed command.

This feature is useful for quickly repeating long commands without typing them again.

2.3. Built-in and External Commands

The following commands were executed:

```
type cd
type ls
type echo
which ls
echo $PATH
echo $PATH | tr ':' '\n'
nosuchprog ; echo "exit code: $?"
```

The `type` command determines how Bash interprets a command. The `which` command searches the directories listed in `PATH` for an executable. The `PATH` variable contains a list of directories separated by colons.

When an external command is entered without an explicit path, the shell searches these directories.

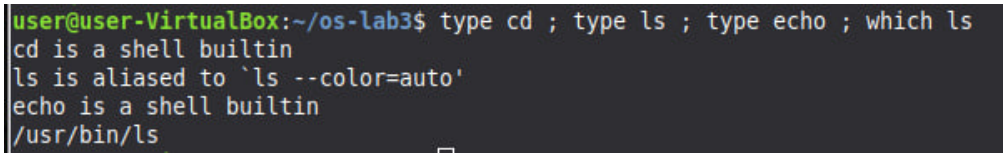


Figure 2.2: Output of the type and which commands

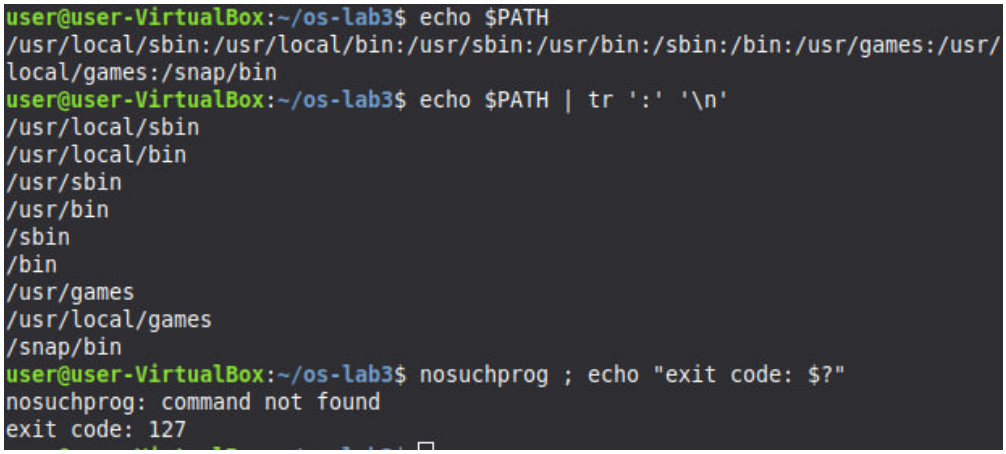


Figure 2.3: The directories contained in PATH

2.4. Finding Help Without a Browser

Several methods for obtaining command documentation were tested:

```
man ls
ls --help | head
man -k "list directory"
help cd
```

The `man` command opens the manual page for a command. Within a manual page, `/sort` searches for the word "sort", `n` moves to the next match, and `q` exits the manual page.

The `-help` option usually provides a shorter command reference. The `help` command is especially useful for Bash built-ins.

2.5. Observe Questions and Answers

1. **Which of cd, ls, echo are built-ins, and why MUST cd be one?**

   `cd` is a Bash built-in because changing the current working directory must change the directory of the current shell process itself. If `cd` were executed as a separate external process, only that child process would change its directory, and the parent shell would remain in the original directory.

   `echo` is commonly available as a Bash built-in, although an external `echo` executable may also exist. `ls` is normally an external executable.

2. **How many directories are in your PATH, and in which one is ls?**

   The exact number of directories depends on the Linux environment. It can be obtained using:

   ```
   echo $PATH | tr ':' '\n' | wc -l
   ```

   The location of `ls` can be obtained using:

   ```
   which ls
   ```

   Prediction: I expect several directories to be listed in `PATH`, and I expect `ls` to be located in one of the standard binary directories, most likely `/usr/bin/ls`.

   Observed result: *[Insert the actual number and path from the terminal.]*

3. **What exit code does a command that is not found return?**

   A command that Bash cannot find normally returns exit code `127`. Therefore:

   ```
   nosuchprog ; echo "exit code: $?"
   ```

   should produce an error followed by:

   ```
   exit code: 127
   ```

4. **Which option of ls sorts by modification time?**

   The `-t` option sorts files by modification time, with the most recently modified files appearing first.

3. Part 2. What the Shell Does to Your Line

3.1. Preparing the Test Directory

The following directory and files were created:

```
mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
```

The resulting directory contained C source files, text files, and a file whose name contained a space.

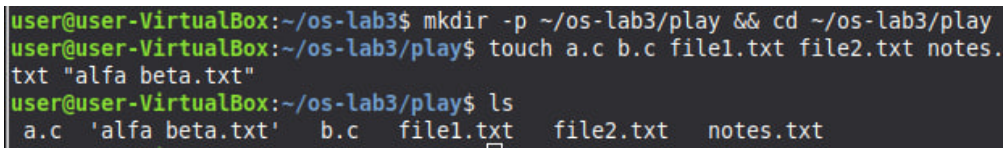


Figure 3.1: Files created in the play directory

3.2. Predictions and Shell Expansion

The following commands were tested:

```
echo *.c
echo file?.txt
echo [a-b]*
echo *.xyz
ls alfa beta.txt
ls "alfa beta.txt"
echo '$HOME'
echo "$HOME"
echo \$HOME
echo {1..5}
echo $((7 * 6))
echo "today is $(date +%A), I am $(whoami)"
touch log{1..3}.txt && ls log*
```

The shell performs several expansions before a command is executed. These include pathname expansion (globbing), parameter expansion, command substitution, arithmetic expansion, brace expansion, and quote processing.

3.3. Prediction Table

The following table records the predictions that can be made before executing the commands and the expected results.

Command	**My prediction**	**Expected result / explanation**
`echo *.c`	The shell will replace `*.c` with all matching C files.	Expected: `a.c b.c`
`echo file?.txt`	The `?` will match exactly one character.	Expected: `file1.txt file2.txt`
`echo [a-b]*`	The pattern will match filenames beginning with either `a` or `b`.	Expected: `a.c b.c`
`echo *.xyz`	There are no matching `.xyz` files, so the pattern will probably remain unchanged.	Expected: `*.xyz`
`ls alfa beta.txt`	The filename containing a space will be interpreted as two separate arguments.	Expected: an error because `alfa` and `beta.txt` do not exist separately.
`ls "alfa beta.txt"`	The quotes will preserve the space as part of the filename.	Expected: the file `alfa beta.txt` will be listed.
`echo '$HOME'`	Single quotes will prevent variable expansion.	Expected: literal `$HOME`
`echo "$HOME"`	Double quotes will still allow parameter expansion.	Expected: the value of the `HOME` variable.
`echo $HOME`	The backslash will prevent the dollar sign from starting variable expansion.	Expected: literal `$HOME`
`echo {1..5}`	Brace expansion will generate a sequence of numbers.	Expected: `1 2 3 4 5`
`echo $((7 * 6))`	Arithmetic expansion will calculate the expression.	Expected: `42`

`echo "today is $(date +%A), I am $(whoami)"`	Command substitution will execute both commands and insert their results.	Expected: the current weekday and current username.
`touch log{1..3}.txt && ls log*`	Brace expansion will create three filenames, and `ls` will then list them.	Expected: `log1.txt log2.txt log3.txt`

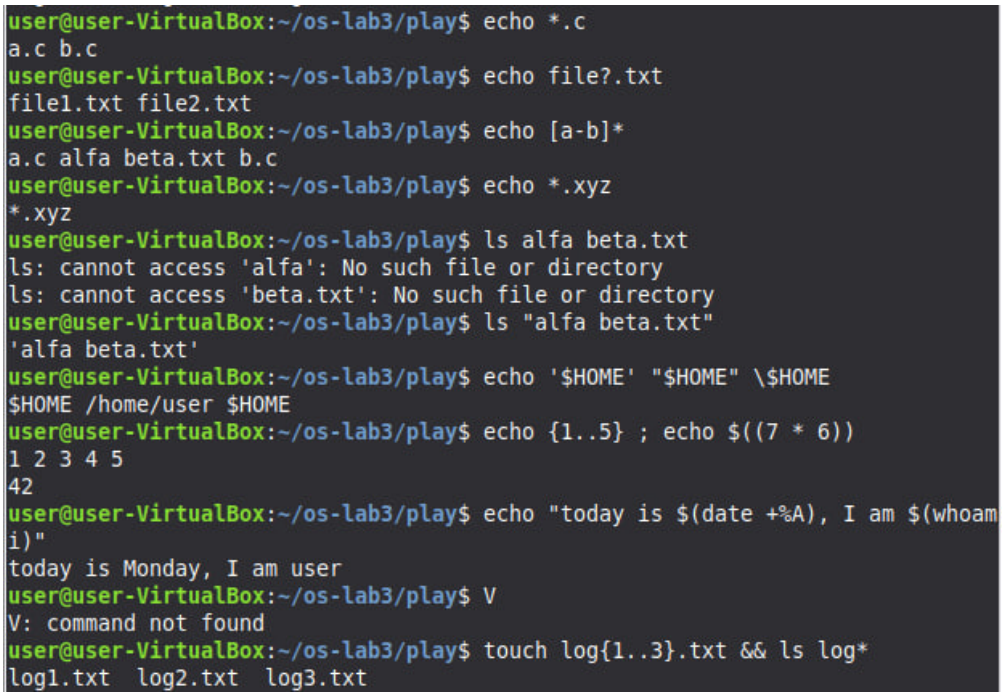


Figure 3.2: Globbing and shell expansion results

3.4. The Filename Containing a Space

The command:

```
ls alfa beta.txt
```

does not refer to the file named `alfa beta.txt`. Instead, the shell passes two separate arguments to `ls`:

```
alfa beta.txt
```

Two ways to correctly access the file are:

```
ls "alfa beta.txt"
ls 'alfa beta.txt'
```

Another possible solution is escaping the space:

```
ls alfa\ beta.txt
```

```
user@user-VirtualBox:~/os-lab3/play$ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
user@user-VirtualBox:~/os-lab3/play$ ls "alfa beta.txt"
'alfa beta.txt'
```

Figure 3.3: The ls error caused by a filename containing a space and its fix

3.5. Single Quotes, Double Quotes and Backslash

Single quotes prevent shell expansion inside them:

```
echo '$HOME'
```

prints the literal string `$HOME`.

Double quotes preserve the argument as one word while still allowing expansions:

```
echo "$HOME"
```

expands the `HOME` variable.

A backslash can prevent the special meaning of the following character:

```
echo \$HOME
```

prints the literal `$HOME`.

The `ls` command may display a filename containing a space in quotes so that the boundaries of the filename are visually clear.

```
user@user-VirtualBox:~/os-lab3/play$ echo '$HOME' "$HOME" \$HOME
$HOME /home/user $HOME
```

Figure 3.4: Single quotes, double quotes and backslash

3.6. Observe Questions and Answers

1. **For each line: your prediction, the real output, and one sentence if they differ.**

   The required commands and their predictions are provided in the prediction table above. The predictions were made before executing the commands according to the laboratory's "Golden rule".

2. **echo knows nothing about files, yet echo *.c printed file names. Who produced them?**

   The Bash shell produced the filenames. Before executing `echo`, Bash performs pathname expansion and replaces the pattern `*.c` with the names of files matching the pattern. Therefore, `echo` receives the filenames as ordinary arguments and does not perform the file search itself.

3. **Why did ls alfa beta.txt fail? Give two different ways to fix it.**

   It failed because the shell interprets the space as an argument separator. Consequently, `ls` receives two arguments instead of the single filename `alfa beta.txt`.

   Two fixes are:

   ```
   ls "alfa beta.txt"
   ls alfa\ beta.txt
   ```

   Single quotes can also be used:

   ```
   ls 'alfa beta.txt'
   ```

4. **What is the difference between single and double quotes?**

   Single quotes prevent shell expansion of their contents. Double quotes prevent word splitting and pathname expansion while still allowing parameter expansion, command substitution, and arithmetic expansion.

4. Part 3. Redirection, Chaining and Jobs

4.1. Standard Output and Standard Error

The following commands were executed:

```
cd ~/os-lab3
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt
cat err.txt
```

The command `ls` receives two paths. The valid `/etc` directory produces normal output, while `/nope` produces an error.

The operator:

```
> out.txt
```

redirects standard output (file descriptor 1) to `out.txt`.

The operator:

```
2> err.txt
```

redirects standard error (file descriptor 2) to `err.txt`.

```
user@user-VirtualBox:~/os-lab3/play$ cd ~/os-lab3
user@user-VirtualBox:~/os-lab3$ ls /etc /nope > out.txt 2> err.txt
user@user-VirtualBox:~/os-lab3$ wc -l out.txt ; cat err.txt
254 out.txt
ls: cannot access '/nope': No such file or directory
user@user-VirtualBox:~/os-lab3$ ls /etc /nope > all.txt 2>&1 ; tail -2 all.tx
t
xml
zsh_command_not_found
user@user-VirtualBox:~/os-lab3$ echo first > notes.md ; echo second >> notes.
md ; cat notes.md
first
second
user@user-VirtualBox:~/os-lab3$ v
v: command not found
user@user-VirtualBox:~/os-lab3$ wc -l < access.log
500
user@user-VirtualBox:~/os-lab3$ ls /nope 2> /dev/null || echo "failed with co
de $?"
failed with code 2
user@user-VirtualBox:~/os-lab3$ mkdir -p tmp && cd tmp && echo "I am in $(pwd
)" ; cd ~/os-lab3
I am in /home/user/os-lab3/tmp
user@user-VirtualBox:~/os-lab3$ sleep 100 &
[1] 4164
user@user-VirtualBox:~/os-lab3$ jobs ; kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
```

Figure 4.1: Standard output and standard error redirected into separate files

4.2. Order of Redirections

The following two commands were compared:

```
ls /etc /nope > all.txt 2>&1
ls /etc /nope 2>&1 > all.txt
```

The first command redirects standard output to `all.txt` and then makes standard error point to the same destination as standard output.

The second command first makes standard error point to the current standard output, which is still the terminal, and only afterwards redirects standard output to `all.txt`. Therefore, the order of redirections matters.

```
user@user-VirtualBox:~/os-lab3$ ls /etc /nope > all.txt 2>&1
[1]+  Terminated              sleep 100
user@user-VirtualBox:~/os-lab3$ ls /etc /nope 2>&1 > all.txt
ls: cannot access '/nope': No such file or directory
```

Figure 4.2: Comparison of the order of redirections

4.3. Input Redirection and Chaining

The following commands were tested:

```
echo first > notes.md
echo second >> notes.md
cat notes.md
wc -l < access.log
wc -l access.log
```

The operator `>` creates or replaces a file and redirects standard output to it. The operator `»` appends output to an existing file. The operator `<` redirects standard input from a file.

Therefore:

```
wc -l < access.log
```

prints only the number because `wc` receives the file through standard input and is not given a filename as an argument.

In contrast:

```
wc -l access.log
```

prints both the number and the filename because `access.log` is explicitly passed as an argument.

4.4. The Operators ;, &&, and ||

The following command was tested:

```
ls /nope 2> /dev/null || echo "failed with code $?"
```

The operators have different meanings:

- `;` executes the next command regardless of whether the previous command succeeded or failed.
- `&&` executes the next command only if the previous command succeeds.
- `||` executes the next command only if the previous command fails.

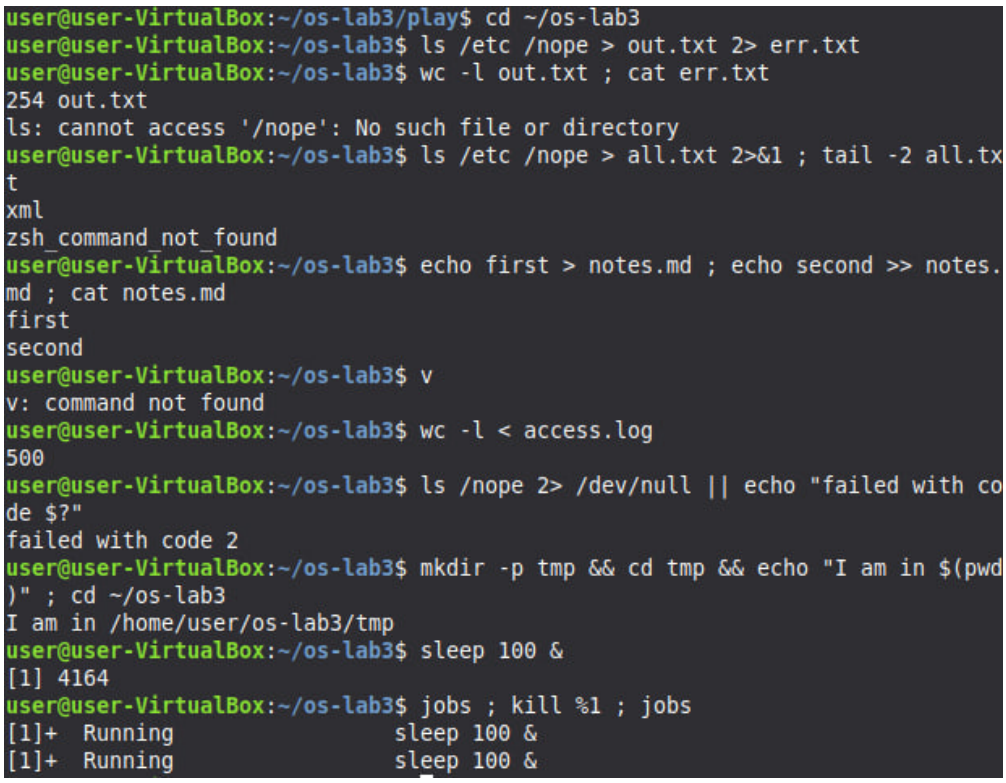


Figure 4.3: The ;, && and || operators

4.5. Background Jobs

The following commands were executed:

```
mkdir -p tmp && cd tmp && echo "I am in $(pwd)"
cd ~/os-lab3
sleep 100 &
jobs
kill %1
jobs
```

The `&` operator starts a command as a background job. The `jobs` command lists jobs belonging to the current shell.

The command:

```
kill %1
```

terminates job number 1.

```
user@user-VirtualBox:~/os-lab3$ sleep 100 &
[1] 4164
user@user-VirtualBox:~/os-lab3$ jobs ; kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
user@user-VirtualBox:~/os-lab3$ ls /etc /nope > all.txt 2>&1
[1]+  Terminated              sleep 100
```

Figure 4.4: Running a background job and stopping it

4.6. Observe Questions and Answers

1. **Which file got the error message, and which got the listing?**

   With:

   ```
   ls /etc /nope > out.txt 2> err.txt
   ```

   the normal directory listing from `/etc` is written to `out.txt`, while the error concerning `/nope` is written to `err.txt`.

2. **What is the difference between > all.txt 2>&1 and 2>&1 > all.txt?**

   In:

   ```
   > all.txt 2>&1
   ```

   standard output is first redirected to `all.txt`. Then standard error is redirected to the current standard output, which is already `all.txt`. Therefore both streams go to the file.

   In:

   ```
   2>&1 > all.txt
   ```

   standard error is first made to point to the original standard output, which is the terminal. Then standard output is redirected to `all.txt`. Consequently, normal output goes to the file while the error remains on the terminal.

3. **Why does wc -l < access.log print only a number?**

   Because `access.log` is used as standard input rather than being supplied as a filename argument. Therefore `wc` has no filename to display.

4. **Explain each operator in one sentence: ;, &&, ||.**

   `;` runs commands sequentially regardless of exit status. `&&` runs the following command only when the previous command succeeds. `||` runs the following command only when the previous command fails.

5. Part 4. Answer Questions with One-Liners

5.1. Dataset

The access log contains 500 requests. Every fifth request is marked `FAIL`; therefore there are:

$$\frac{500}{5} = 100$$

failed requests.

The page is generated using:

```
/page$((i % 4))
```

Therefore the four pages `/page0`, `/page1`, `/page2`, and `/page3` are requested.

5.2. Q1. How Many Requests Failed?

The one-liner is:

```
grep -c FAIL access.log
```

Prediction: Since every fifth request is marked `FAIL`, I expect exactly 100 failed requests.

Expected result:

```
100
```

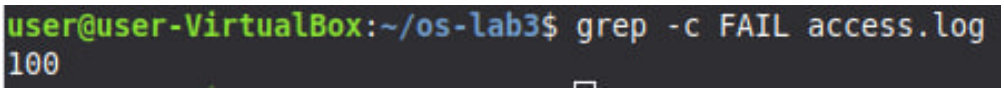


Figure 5.1: Number of failed requests

5.3. Q2. How Many Different Pages Were Requested, and Which?

The one-liner is:

```
cut -d' ' -f5 access.log | sort -u
```

Prediction: The expression `i % 4` can produce 0, 1, 2, and 3, so I expect four different pages.

Expected output:

```
/page0
/page1
/page2
/page3
```

The number of different pages can be obtained with:

```
cut -d' ' -f5 access.log | sort -u | wc -l
```

The expected result is:

```
4
```

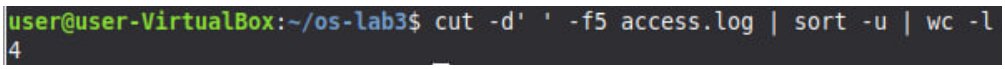


Figure 5.2: Different pages requested in the access log

5.4. Q3. How Many Requests Did Each Page Get?

The one-liner is:

```
cut -d' ' -f5 access.log | sort | uniq -c | sort -k2
```

Prediction: Because the page number is generated using `i % 4`, the 500 requests should be distributed as evenly as possible among four pages.

Since:

$$\frac{500}{4} = 125$$

I expect each page to receive 125 requests.

Expected result:

```
125 /page0
125 /page1
125 /page2
125 /page3
```

The four counts should add up to 500.

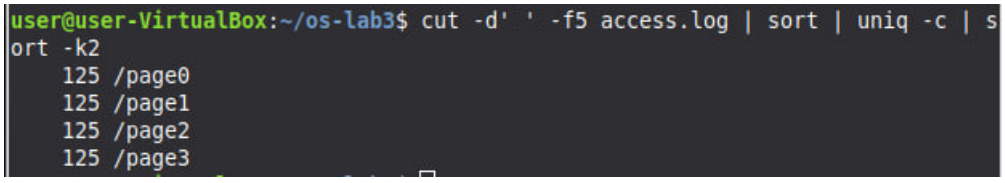


Figure 5.3: Number of requests received by each page

5.5. Q4. Which User or Users Have the Most Failed Requests?

The one-liner is:

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
```

Prediction: Failures occur when the request number is divisible by 5, while users are assigned using `i % 7`. Because 5 and 7 are relatively prime, the failures should be distributed approximately evenly among the seven users.

There are 100 failures. Dividing them among seven users gives approximately:

$$\frac{100}{7} \approx 14.29$$

Therefore I expect some users to have 15 failures and the others to have 14 failures.

For the generated dataset, the users with the maximum of 15 failures are expected to be:

```
user3
user5
```

The full distribution can be checked using the command above.

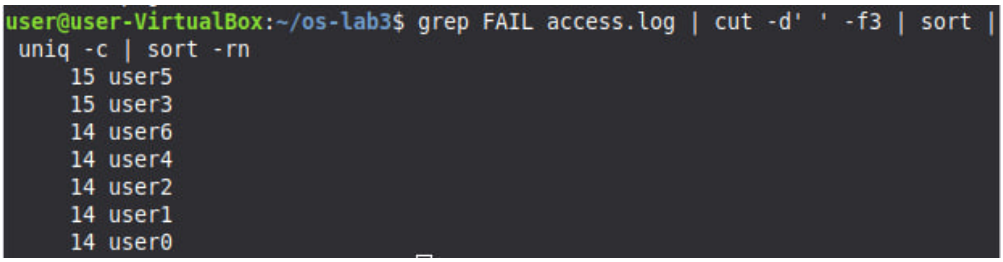


Figure 5.4: Failed requests grouped by user

5.6. Q5. How Many Requests Did user3 Make, and How Many Failed?

The one-liner is:

```
echo "total: $(grep -c ' user3 ' access.log), failed: $(grep ' user3 ' access.log
    | grep -c FAIL)"
```

Prediction: Since users are generated using `i % 7`, user3 should occur approximately one seventh of the time:

$$\frac{500}{7} \approx 71.43$$

Therefore I expect 72 requests for user3. Based on the failure pattern, I expect 15 of them to be failures.

Expected result:

```
total: 72, failed: 15
```

```
user@user-VirtualBox:~/os-lab3$ echo "total: $(grep -c 'user3' access.log), f
ailed: $(grep 'user3' access.log | grep -c 'FAIL')"
total: 72, failed: 15
```

Figure 5.5: Total and failed requests made by user3

5.7. Q6. Print the Last 3 Failed Requests, Showing Only Time and User

The one-liner is:

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
```

Prediction: The last three failed requests should correspond to request numbers 490, 495, and 500 because failures occur every fifth request.

Their times are generated using `i % 60`:

```
490 % 60 = 10
495 % 60 = 15
500 % 60 = 20
```

Their users are generated using `i % 7`:

```
490 % 7 = 0
495 % 7 = 5
500 % 7 = 3
```

Therefore the expected output is:

```
10:10 user0
10:15 user5
10:20 user3
```

```
user@user-VirtualBox:~/os-lab3$ grep FAIL access.log | tail -3 | cut -d' ' -f
2,3
10:10 user0
10:15 user5
10:20 user3
```

Figure 5.6: The last three failed requests showing time and user

5.8. Q7. Which Login Shells Appear in /etc/passwd, and How Many Accounts Use Each?

The required fields are separated by `:`, and the login shell is field 7.

The one-liner is:

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

Prediction: I expect several different login shells to appear, most commonly shells such as `/usr/sbin/nologin`, `/bin/bash`, or `/bin/sync`, depending on the Linux distribution.

The exact shell names and counts depend on the Linux environment used for the laboratory.

Observed output:

```
40 /usr/sbin/nologin
 6 /bin/false
 2 /bin/bash
 1 /bin/sync
```

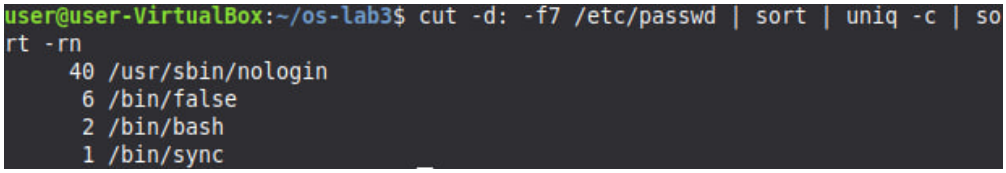


Figure 5.7: Login shells appearing in /etc/passwd and their account counts

5.9. Q8. Why Do ls /etc | wc -l and ls -l /etc | wc -l Differ by Exactly One?

The two commands are:

```
ls /etc | wc -l
ls -l /etc | wc -l
```

Prediction: I expect the second command to produce exactly one more line because the long format of `ls -l` normally begins with a `total` line.

The long format of `ls -l` normally begins with an additional line containing the total number of filesystem blocks used by the displayed entries:

```
total N
```

Therefore, when the output is piped to `wc -l`, `ls -l /etc` produces one additional line compared with the normal `ls /etc` output.

Thus, the difference is exactly one line because of the `total` header.

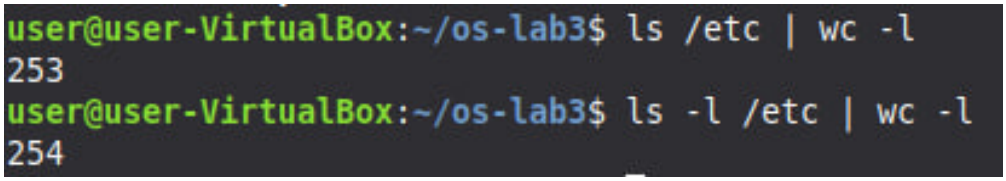


Figure 5.8: Comparison of ls /etc and ls -l /etc line counts

5.10. Complete oneliners.sh Execution

The following is the completed structure of `oneliners.sh`:

```
#!/bin/bash

# Q1. How many requests failed?
grep -c FAIL access.log

# Q2. How many different pages were requested, and which?
cut -d' ' -f5 access.log | sort -u

# Q3. How many requests did each page get?
cut -d' ' -f5 access.log | sort | uniq -c | sort -k2

# Q4. Which user or users have the most failed requests, and how many?
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn

# Q5. How many requests did user3 make, and how many of them failed?
echo "total: $(grep -c ' user3 ' access.log), failed: $(grep ' user3 ' access.log
    | grep -c FAIL)"

# Q6. Print the last 3 failed requests, showing only time and user.
grep FAIL access.log | tail -3 | cut -d' ' -f2,3

# Q7. Which login shells appear in /etc/passwd, and how many accounts use each?
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn

# Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?
echo "ls /etc has only directory entries, while ls -l /etc also prints the total
    header."
```

The script was executed with:

```
bash oneliners.sh
```

The main consistency checks are:

- the access log contains exactly 500 lines;
- Q1 reports 100 failures;
- the four page counts in Q3 add up to 500;
- the failed-request counts per user add up to 100.

```
    125 /page3
user@user-VirtualBox:~/os-lab3$ grep FAIL access.log | cut -d' ' -f3 | sort |
 uniq -c | sort -rn
     15 user5
     15 user3
     14 user6
     14 user4
     14 user2
     14 user1
     14 user0
user@user-VirtualBox:~/os-lab3$ echo "total: $(grep -c 'user3' access.log), f
ailed: $(grep 'user3' access.log | grep -c 'FAIL')"
total: 72, failed: 15
user@user-VirtualBox:~/os-lab3$ grep FAIL access.log | tail -3 | cut -d' ' -f
2,3
10:10 user0
10:15 user5
10:20 user3
user@user-VirtualBox:~/os-lab3$ cut -d: -f7 /etc/passwd | sort | uniq -c | so
rt -rn
     40 /usr/sbin/nologin
      6 /bin/false
      2 /bin/bash
      1 /bin/sync
user@user-VirtualBox:~/os-lab3$ ls /etc | wc -l
253
user@user-VirtualBox:~/os-lab3$ ls -l /etc | wc -l
254
user@user-VirtualBox:~/os-lab3$
```

Figure 5.9: Full execution of oneliners.sh

5.11. Bonus: Failed Requests at Minutes Ending in Zero

The bonus task can be solved using:

```
grep -E '10:[0-5][0] .* FAIL ' access.log
```

To count the matching requests:

```
grep -E '10:[0-5][0] .* FAIL ' access.log | wc -l
```

Prediction: Failed requests occur every five requests. The minutes ending in zero correspond to requests whose generated minute is 00, 10, 20, 30, 40, or 50. Among these, only some are failures.

The failed requests satisfying this condition are expected to be:

`60, 120, 180, 240, 300, 360, 420, 480`

Therefore the expected number is:

8

6. Part 5. Your First Script

6.1. Creating report.sh

The purpose of `report.sh` is to turn the previously developed command-line operations into a reusable Bash program. The completed script is:

```
#!/bin/bash

# usage: ./report.sh access.log

log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")

echo "$total requests, $fails failed"

for user in user0 user1 user2 user3 user4 user5 user6
do
    count=$(grep FAIL "$log" | grep -c " $user ")
    echo "$user: $count failures"
done
```

The script checks whether the supplied file exists. If the file is missing, an error is written to standard error and the script exits with status code 1. If the file exists, the script prints the total number of requests, the number of failures, and the failure count for each user.

```bash
#!/bin/bash
# usage: ./report.sh access.log

log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")

echo "$total requests, $fails failed"

for user in user0 user1 user2 user3 user4 user5 user6
do
    count=$(grep FAIL "$log" | grep -c " $user ")
    echo "$user: $count failures"
done
```

Figure 6.1: Contents of report.sh

6.2. Running the Script

The script was made executable using:

```
chmod +x report.sh
```

It was then executed using:

```
./report.sh access.log
```

For the generated 500-line dataset, the beginning of the output is:

```
500 requests, 100 failed
```

The user failure counts are printed afterwards.

```
user@user-VirtualBox:~/os-lab3$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 6.2: Running report.sh with access.log

6.3. Testing Error Handling

The following command was used:

```
./report.sh nope.log ; echo "exit code: $?"
```

Because `nope.log` does not exist, the script prints an error message to standard error and exits with code 1.

Expected result:

```
no such file: nope.log
exit code: 1
```

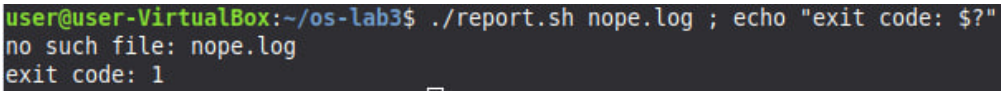


Figure 6.3: Error handling when the input file does not exist

6.4. Observe Questions and Answers

1. **What does the first line #!/bin/bash do, and what happens if you run before chmod +x?**

   The first line is called a shebang. It specifies that the script should be interpreted using the Bash shell.

   Without executable permission, running:

   ```
   ./report.sh access.log
   ```

   normally results in a permission error.

   The script can still be executed explicitly through Bash:

   ```
   bash report.sh access.log
   ```

   because Bash is then directly instructed to interpret the file.

2. **Why must you type ./report.sh and not just report.sh?**

   The current directory is normally not included in the shell's `PATH` for security reasons. Therefore Bash does not automatically search the current directory for commands.

   The prefix:

   ```
   ./
   ```

explicitly tells Bash to execute the file located in the current directory.

3. **Why does the script write the error with &>2 and end with exit 1?**

   The expression:

   `>&2`

   redirects the error message to standard error rather than standard output.

   The command:

   `exit 1`

   terminates the script with a non-zero exit status, indicating that an error occurred.

7. Conclusions

The laboratory work provided practical experience with the Linux command line and Bash shell. The first part demonstrated how command completion, command history, built-in commands, external executables, the `PATH` variable, and manual pages can be used to work efficiently in a terminal.

The second part demonstrated that the shell processes a command line before executing the selected program. Pathname expansion, brace expansion, parameter expansion, command substitution, arithmetic expansion, quoting, and escaping all affect the arguments eventually passed to a command. In particular, the experiment with `echo *.c` demonstrated that filename expansion is performed by the shell rather than by `echo` itself.

The third part showed how standard input, standard output, and standard error can be redirected independently. The experiments with `>`, `>>`, `<`, and `2>&1` demonstrated that the order of redirections is significant. Command chaining with `;`, `&&`, and `||` also demonstrated how commands can be conditionally executed. Background jobs were managed using `&`, `jobs`, and `kill`.

The fourth part demonstrated the usefulness of Unix text filters. A 500-line web access log was analyzed using pipelines composed of `grep`, `cut`, `sort`, `uniq`, `wc`, `head`, and `tail`. The tasks showed that relatively small commands can be combined to answer practical questions about structured data.

The fifth part extended these techniques into Bash scripting. The `report.sh` script accepted a log file as an argument, validated the input, calculated statistics, and returned a meaningful error code when the file did not exist. This demonstrated how interactive command-line operations can be converted into reusable programs.

Overall, the laboratory demonstrated that effective command-line work depends not only on knowing individual commands but also on understanding how the shell transforms commands, connects processes, redirects data, and combines simple utilities into larger solutions.

8. Appendix A. oneliners.sh

```
#!/bin/bash

# Q1. How many requests failed?
grep -c FAIL access.log

# Q2. How many different pages were requested, and which?
cut -d' ' -f5 access.log | sort -u

# Q3. How many requests did each page get?
cut -d' ' -f5 access.log | sort | uniq -c | sort -k2

# Q4. Which user or users have the most failed requests, and how many?
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn

# Q5. How many requests did user3 make, and how many of them failed?
echo "total: $(grep -c ' user3 ' access.log), failed: $(grep ' user3 ' access.log
    | grep -c FAIL)"

# Q6. Print the last 3 failed requests, showing only time and user.
grep FAIL access.log | tail -3 | cut -d' ' -f2,3

# Q7. Which login shells appear in /etc/passwd, and how many accounts use each?
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn

# Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?
echo "ls /etc has only directory entries, while ls -l /etc also prints the total
   header."
```

9. Appendix B. report.sh

```
#!/bin/bash

# usage: ./report.sh access.log

log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
```

```
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")

echo "$total requests, $fails failed"

for user in user0 user1 user2 user3 user4 user5 user6
do
    count=$(grep FAIL "$log" | grep -c " $user ")
    echo "$user: $count failures"
done
```